MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory works no. 5 and 6

Operating Systems

Topic: The File System – from Linux to xv6

Done by:
st. gr. FAF-242, **Vasile Brînză**

Checked by:
Reițman Patricia, *university assistant*

Chișinău 2026

Table of Contents

PURPOSE OF THE WORK **3**
1 LABORATORY WORK NO. 5 – THE FILE SYSTEM WITH YOUR OWN HANDS (LINUX) **4**
1.1 The tree and the file types 4
1.2 Inodes and links 5
1.3 Permissions 7
1.4 Disks and the kernel as files 9
2 LABORATORY WORK NO. 6 – INSIDE THE XV6 FILE SYSTEM **11**
2.1 Building the new programs 11
2.2 Part 1. Reading what xv6 shows 11
2.3 Part 2. The program mystat 12
2.4 Part 3. Measuring the limits 15
2.5 Part 4. The program find 16
CONCLUSIONS **19**
REFERENCES **20**

PURPOSE OF THE WORK

The purpose of laboratory works no. 5 and no. 6 is to understand how a Unix file system is organised, first by observing it from the outside on a real Linux system, then by reading and extending the much smaller file system of the xv6 teaching operating system. By the end of the work the goals are:

- to recognise the file types of the Linux directory tree (regular files, directories, symbolic links, character and block devices) and to read them from the output of `ls -l`;
- to understand the inode as the real object behind a file name, and to explain the behaviour of hard links, symbolic links, directory link counts and sparse files;
- to apply and explain Unix permissions: octal modes, `umask`, the meaning of `r` and `x` on a directory, and the special SetUID and sticky bits;
- to inspect disks, partitions and mounted file systems, and to read kernel information exposed as files in `/proc`;
- to decode the on-disk structures of xv6 (`struct dirent`, `struct dinode`) and to measure the limits written in `kernel/fs.h`;
- to write two user programs for xv6 in C: `mystat`, which prints the metadata returned by `stat()`, and `find`, which walks the directory tree recursively.

Laboratory work no. 5 (Track A) was done on a Lenovo laptop running Linux, as a normal user (`kynex`), in the folder `~/os-2026-VasileBrinza/lab5`. Laboratory work no. 6 (Track B) was done in the `~/xv6-riscv` tree at the course revision `06aad25`, compiled with the RISC-V cross compiler (`riscv64-linux-gnu-gcc`) and run in the QEMU emulator (`make qemu`). As allowed by the lab assignment, both works are presented in a single report, with a separate chapter for each laboratory work. The source code is pushed to the Git repository of the course [5].

1 LABORATORY WORK NO. 5 – THE FILE SYSTEM WITH YOUR OWN HANDS (LINUX)

All commands of this chapter were run in bash on the Linux host, as the normal user `kynex` (not root, because root bypasses most permission checks and the experiments of Part 3 would give misleading results).

1.1 The tree and the file types

The first group of commands lists several important paths of the tree, the first entries of `/dev`, the type of some files as detected by `file`, and the disk usage of two large directories.

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Mar 15  2025 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 19 16:05 /dev/null
drwxr-xr-x 151 root root 12288 Oct 11 23:33 /etc
-rw-r--r--   1 root root  2834 Mar 15  2025 /etc/passwd
drwxrwxrwt  19 root root  4096 Oct 19 16:21 /tmp

$ ls -l /dev | head
total 0
drwxr-xr-x  2 root root      60 Oct 19 16:05 accel
crw-r--r--  1 root root 10, 235 Oct 19 16:05 autofs
drwxr-xr-x  2 root root     320 Oct 19 16:05 block
crw-rw----  1 root disk 10, 234 Oct 19 16:05 btrfs-control
drwxr-xr-x  3 root root      60 Oct 19  2026 bus
drwxr-xr-x  2 root root    5260 Oct 19 16:05 char
crw--w----  1 root tty   5,   1 Oct 19 16:05 console
lrwxrwxrwx  1 root root      11 Oct 19 16:05 core -> /proc/kcore
drwxr-xr-x 18 root root     360 Oct 19 16:05 cpu

$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
             dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
             BuildID[sha1]=3eca7e39..., for GNU/Linux 3.2.0, stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin

$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
519M    /var/log
16G     /usr
```

Listing 1.1 – The tree, the file types and the disk usage

Answers to the Observe questions

1. For each path of the second line, write its type from the first letter of ls -l.

Table 1.1 – File types read from ls -ld

Path	First letter	Type
`/bin`	`l`	symbolic link (to usr/bin)
`/dev/null`	`c`	character device (major 1, minor 3)
`/etc`	`d`	directory
`/etc/passwd`	`-`	regular file
`/tmp`	`d`	directory (with the sticky bit, shown as t)

2. What is /bin on your system: a directory or a link, and to where?

`/bin` is a symbolic link to `usr/bin` (a relative target, i.e. `/usr/bin`). Both `ls -ld` (first letter `l` and the arrow `-> usr/bin`) and `file` ("symbolic link to usr/bin") confirm it. This is the "merged /usr" layout of modern distributions: the programs live only in `/usr/bin`, and `/bin` is kept as a link for compatibility with old scripts that use `#!/bin/sh`.

3. Find one character device and one block device in /dev.

Character device: `/dev/null` (`crw-rw-rw-`, major 1, minor 3); others from the listing are `/dev/console` (5, 1) and `/dev/autofs` (10, 235). Character devices are accessed as a stream of bytes, without the block cache. Block device: the NVMe disk `/dev/nvme0n1` (major 259, minor 0, see `lsblk` in Listing 1.6) and its partitions `/dev/nvme0n1p1 … p5`; with `ls -l` its line starts with `b`. A block device is accessed in fixed-size blocks through the kernel buffer cache, which is what file systems need.

4. Which is bigger on your laptop, /var/log or /usr, and by roughly how much?

`/usr` is much bigger: 16 GB against 519 MB for `/var/log`, a difference of about 15.5 GB, i.e. `/usr` is roughly 30 times bigger. `/usr` holds all installed programs and libraries, while `/var/log` holds only log files.

1.2 Inodes and links

The experiments of this part create a file, give it a second name with a hard link, remove the original name, create a symbolic link and remove its target, then look at directory link counts and at the difference between the size of a file and the space it occupies.

```
$ echo hello > notes.txt && stat notes.txt && ls -li notes.txt
  File: notes.txt
  Size: 6             Blocks: 8          IO Block: 4096   regular file
Device: 259,5   Inode: 12455072    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kynex)   Gid: ( 1000/   kynex)
Access: 2026-10-19 16:36:08.897020415 +0300
Modify: 2026-10-19 16:36:08.897020415 +0300
Change: 2026-10-19 16:36:08.897020415 +0300
 Birth: 2026-10-19 16:36:08.897020415 +0300
12455072 -rw-rw-r-- 1 kynex kynex 6 Oct 19 16:36 notes.txt

$ ln notes.txt hard.txt && ls -li notes.txt hard.txt
12455072 -rw-rw-r-- 2 kynex kynex 6 Oct 19 16:36 hard.txt
12455072 -rw-rw-r-- 2 kynex kynex 6 Oct 19 16:36 notes.txt

$ rm notes.txt
$ cat hard.txt && ls -li hard.txt
hello
12455072 -rw-rw-r-- 1 kynex kynex 6 Oct 19 16:36 hard.txt
```

Listing 1.2 – An inode and a hard link

```
$ ln -s hard.txt soft.txt && ls -li hard.txt soft.txt
12455072 -rw-rw-r-- 1 kynex kynex 6 Oct 19 16:36 hard.txt
12455074 lrwxrwxrwx 1 kynex kynex 8 Oct 19 16:36 soft.txt -> hard.txt
$ cat soft.txt
hello
$ rm hard.txt
$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 kynex kynex 8 Oct 19 16:36 soft.txt -> hard.txt

$ mkdir d ; ls -ld d
drwxrwxr-x 2 kynex kynex 4096 Oct 19 16:37 d
$ mkdir d/x d/y && ls -ld d
drwxrwxr-x 4 kynex kynex 4096 Oct 19 16:38 d

$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 kynex kynex 2 Oct 19 16:38 one.txt
4.0K    one.txt
$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 kynex kynex 1.0G Oct 19 16:39 big.img
0       big.img
$ rm one.txt big.img soft.txt
```

Listing 1.3 – Symbolic link, directory links and sparse file

Answers to the Observe questions

1. Which number in stat is the inode, and which tells how many names the file has?

The inode number is `Inode: 12455072` (the same number is the first column of `ls -li`). The number of names is the link counter `Links: 1` (the second column of `ls -l`). After `ln notes.txt hard.txt` the counter became 2 and both names showed the same inode 12455072: there is one file with two names.

2. After rm notes.txt, why could you still read hard.txt? What does rm really remove?

`rm` does not delete a file, it deletes a name: it calls `unlink()`, which removes the directory entry "notes.txt → 12455072" and decrements the link counter of the inode (2 → 1). The inode and its data blocks are freed only when the link counter reaches 0 and no process has the file open. Because `hard.txt` still pointed to inode 12455072, the content "hello" was intact.

3. Why did cat soft.txt fail at the end, although soft.txt still exists? How big is soft.txt and why exactly that size?

A symbolic link is a separate file (its own inode, 12455074) whose content is only a path. `cat soft.txt` makes the kernel follow the path "hard.txt"; after `rm hard.txt` that name no longer exists, so the result is "No such file or directory" – a dangling link. The link itself still exists, as `ls -l` shows. Its size is 8 bytes, exactly the length of the stored path `hard.txt` (8 characters, no terminating zero).

4. A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.

A directory is referenced by name from its parent and by special entries inside the tree:

- link 1 – the entry `d` in the parent directory `lab5`;
- link 2 – the entry `.` inside `d` itself;
- links 3 and 4 – the entry `..` inside each subdirectory, `d/x/..` and `d/y/..`.

In general, the link count of a directory on ext4 is 2 + the number of its subdirectories.

5. Why does a 2-byte file use 4.0K, and a 1 GB file use 0?

`ls -l` shows the logical size (the number of bytes in the file), while `du` shows the space actually allocated on disk. ext4 allocates space in whole blocks of 4096 bytes, so the 2-byte file `one.txt` ("x" and a newline) occupies one full 4 KiB block. `truncate -s 1G` only sets the size field of the inode and writes no data, so no block is allocated: `big.img` is a sparse file, and reading it returns zeros for the "holes". Blocks are allocated only when data is really written.

1.3 Permissions

The permissions of a file, the effect of `umask`, the meaning of `r` and `x` on a directory and the special bits were tested with the following commands.

```
$ echo secret > s.txt
$ chmod 600 s.txt && ls -l s.txt
-rw------- 1 kynex kynex 7 Oct 19 16:39 s.txt
$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt && cat s.txt
secret
$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
drwxrwxr-x 2 kynex kynex 4096 Oct 19 16:41 newdir
-rw-rw-r-- 1 kynex kynex    0 Oct 19 16:41 new.txt
```

Listing 1.4 – File permissions and umask

```
$ mkdir box && echo hi > box/f
$ chmod u=r,go= box ; ls box ; cat box/f          # read only
ls: cannot access 'box/f': Permission denied
f
cat: box/f: Permission denied
$ chmod u=x,go= box ; ls box ; cat box/f          # execute only
ls: cannot open directory 'box': Permission denied
hi
$ chmod 755 box && rm -rf box

$ ls -l /usr/bin/passwd /etc/shadow ; ls -ld /tmp
-rw-r----- 1 root shadow  1344 Apr 29  2025 /etc/shadow
-rwsr-xr-x 1 root root   64152 May 30  2024 /usr/bin/passwd
drwxrwxrwt 20 root root   4096 Oct 19 16:29 /tmp
```

Listing 1.5 – Directory permissions and special bits

Answers to the Observe questions

1. Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.

Table 1.2 – Common octal modes

Mode	rwx string	Real use
600	rw-------	private files: SSH private key ~/.ssh/id_ed25519, s.txt above
644	rw-r--r--	public, read-only files: /etc/passwd, web pages, documents
755	rwxr-xr-x	programs and normal directories: /usr/bin/ls, /usr
700	rwx------	private directories and scripts: ~/.ssh, /root

Each octal digit is the sum r = 4, w = 2, x = 1, for the owner, the group and the others, in this order.

2. With chmod 000 you own the file but cannot read it. Why does ownership not help?

The kernel chooses exactly one class of permission bits: if the process user ID equals the owner of the file, only the owner bits are checked; otherwise, if a group matches, only the group bits; otherwise the "others" bits. With mode 000 the owner bits are `---`, so the read is refused, and the kernel does not fall back to the group or other bits. Ownership gives only one extra right: the owner may always run `chmod`, which is why `chmod u+r s.txt` worked and the file became readable again.

3. What is your umask, and how does it explain the permissions of new.txt and newdir?

The umask is `0002`. It lists the bits that are removed from the default mode requested by the program (mode AND NOT umask). `touch` creates files with mode 666 (`rw-rw-rw-`); removing the "write for others" bit gives 664, `-rw-rw-r--` (new.txt). `mkdir` creates directories with mode 777; removing the same bit gives 775, `drwxrwxr-x` (newdir). Files do not get `x` because the default mode for files never contains it.

4. Explain the box experiment: what does r allow on a directory, and what does x allow?

A directory is a table of (name, inode) entries. **r** allows reading that table, i.e. listing the names: with `u=r` the command `ls box` printed the name `f` (the error line comes from `ls` trying to `stat` the entry to get its details), but `cat box/f` was refused because the path cannot be traversed. **x** allows searching the directory, i.e. using it in a path to reach an entry whose name is already known: with `u=x` the command `ls box` failed, but `cat box/f` printed "hi". To list a directory and open the files in it, both `r` and `x` are needed (mode 755 or 700).

5. What do the s in /usr/bin/passwd and the t in /tmp mean, and why does passwd need the s?

The `s` in the owner execute position of `/usr/bin/passwd` is the SetUID bit: the program runs with the effective user ID of its owner (root), not of the user who started it. passwd needs it because the password

hashes are stored in `/etc/shadow`, which has mode `rw-r-----` and owner `root:shadow`, so a normal user can neither read it (`cat /etc/shadow` gives "Permission denied") nor write it. Thanks to SetUID, passwd can update the user's own entry, and the program itself makes sure that a user can change only their own password. The `t` in `/tmp` (`drwxrwxrwt`) is the sticky bit: everybody may create files in `/tmp`, but a file can be deleted or renamed only by its owner, the owner of the directory, or root.

1.4 Disks and the kernel as files

```
$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 953.9G  0 disk
├─nvme0n1p1 259:1    0   100M  0 part /boot/efi
├─nvme0n1p2 259:2    0    16M  0 part
├─nvme0n1p3 259:3    0 757.8G  0 part
├─nvme0n1p4 259:4    0   759M  0 part
└─nvme0n1p5 259:5    0 195.2G  0 part /
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p5  192G   93G   89G  51% /
$ mount | grep " / "
/dev/nvme0n1p5 on / type ext4 (rw,relatime,errors=remount-ro)
$ grep -c processor /proc/cpuinfo
16
$ head -3 /proc/meminfo
MemTotal:       28533320 kB
MemFree:        19383828 kB
MemAvailable:   23101732 kB
$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   6450
Ngid:   0
```

Listing 1.6 – Disks, partitions and files in /proc

Answers to the Observe questions

1. Draw your disk: disk name, partitions, which one is mounted at / and with what file system type.

The laptop has one NVMe SSD, `/dev/nvme0n1` (953.9 GB), with a GPT partition table and five partitions (Table 1.3, Figure 1.1). The root file system `/` is on `/dev/nvme0n1p5`, formatted as **ext4** (from `mount`). The partitions p2–p4 are not mounted in Linux; by their sizes they are the partitions of a dual-boot Windows installation.

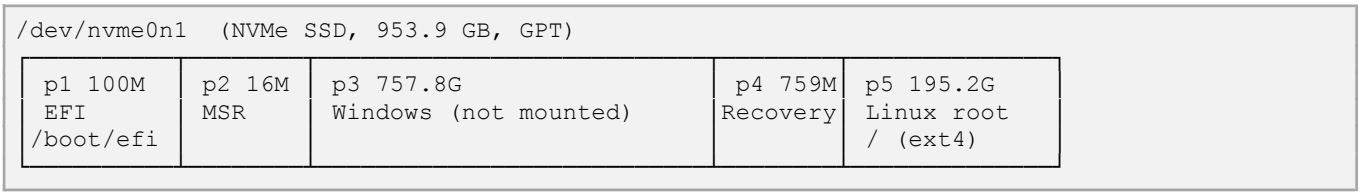


Figure 1.1 – Layout of the disk /dev/nvme0n1

Table 1.3 – Partitions of /dev/nvme0n1

Partition	Size	Mount point	Role / file system
nvme0n1p1	100 MB	/boot/efi	EFI system partition (FAT32)
nvme0n1p2	16 MB	–	Microsoft reserved partition
nvme0n1p3	757.8 GB	–	Windows system partition
nvme0n1p4	759 MB	–	Windows recovery partition
nvme0n1p5	195.2 GB	/	Linux root, ext4

Note: `lsblk` without `-f` does not print the file system type, so the roles of p1–p4 are deduced from their size and mount point; only the type of `/` (ext4) is read directly from `mount`.

2. How full is your root file system?

According to `df -h /`, the root file system on `/dev/nvme0n1p5` has 192 GB, of which 93 GB are used and 89 GB are available: it is **51 %** full. (Used + available is smaller than the size because ext4 reserves about 5 % of the blocks for root.)

3. How many CPU cores and how much RAM does your laptop have, according to /proc?

`/proc/cpuinfo` lists **16** logical processors (hardware threads). `/proc/meminfo` reports `MemTotal: 28533320 kB`, i.e. about **27.2 GiB** of RAM usable by the kernel (the installed RAM minus the memory reserved by the firmware and the integrated GPU).

4. /proc/self/status showed the name of a program. Which one, and why that one?

It showed `Name: head`. `/proc/self` is a "magic" symbolic link that the kernel resolves to `/proc/<PID>` of the process that is looking at it. The process that opened `/proc/self/status` was `head` itself (PID 6450), not the shell, so the file described `head`. This shows that `/proc` is not stored on disk: its files are generated by the kernel at the moment they are read.

2 LABORATORY WORK NO. 6 – INSIDE THE XV6 FILE SYSTEM

In this chapter the same ideas – inodes, directories as tables of entries, hard links – are studied inside xv6, whose whole file system is only a few hundred lines of C. All xv6 commands were typed at the xv6 shell prompt `$` inside QEMU (`make qemu`, exit with Ctrl-A X); the build commands were run in the host terminal in `~/xv6-riscv`.

2.1 Building the new programs

Every new user program is a file `user/NAME.c` and a line `$U/_NAME\` in the `UPROGS` list of the Makefile (a TAB at the start and nothing after the backslash). The two programs of this lab were added after `_zombie`:

```
UPROGS=\
    $U/_cat\
    ...
    $U/_wc\
    $U/_zombie\
    $U/_mystat\
    $U/_find\
    $U/_logstress\
    ...
```

Listing 2.1 – The UPROGS lines added to the Makefile

2.2 Part 1. Reading what xv6 shows

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
echo           2 4 34312
...
wc             2 17 36336
zombie         2 18 33648
mystat         2 19 35160
find           2 20 37280
...
sync           2 24 33712
console        3 25 0
$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
```

Listing 2.2 – ls in the root directory and in a new directory

Each line of the xv6 `ls` is: name, type, inode number, size in bytes. The types are defined in `kernel/stat.h`:

```
#define T_DIR    1 // Directory
#define T_FILE   2 // File
#define T_DEVICE 3 // Device
```

Listing 2.3 – File types in kernel/stat.h

Answers to the Observe questions

1. What do types 1, 2 and 3 mean (kernel/stat.h)?

1 = `T_DIR` (directory: `.`, `..`, `a`), 2 = `T_FILE` (regular file: README and all programs), 3 = `T_DEVICE` (device file: `console`, which has size 0 because it has no data blocks – reads and writes go to the console driver selected by the major number).

2. Which inode number does the root directory have, and why are . and .. the same there?

The root directory has inode **1** (`#define ROOTINO 1` in `kernel/fs.h`). In every directory `.` points to the directory itself and `..` to its parent. The root has no parent, so when `mkfs` builds the disk image it makes `..` of the root point to the root itself: both entries contain inode 1, and `cd ..` in `/` stays in `/`. (In the new directory `a`, `.` is inode 26 and `..` is inode 1, the root.)

3. An empty directory a has size 32. How big is one entry, and which two entries does a contain?

A directory entry is `struct dirent { ushort inum; char name[DIRSIZ]; }` = 2 + 14 = **16 bytes**, so 32 bytes are exactly two entries: `.` (inode 26, the directory `a` itself) and `..` (inode 1, the root). The root has size 1024 because its entries fill one whole block of `BSIZE` = 1024 bytes (64 entries); the directory grows one block at a time.

2.3 Part 2. The program mystat

`mystat` calls the library function `stat()` for every argument, which fills a `struct stat` (fields `dev`, `ino`, `type`, `nlink`, `size`). On error it prints a message to file descriptor 2 (standard error) and continues with the next argument. `typename()` translates the numeric type into a word.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *typename(int t) {
  if (t == T_DIR)
    return "directory";
  if (t == T_FILE)
    return "file";
  if (t == T_DEVICE)
    return "device";
  return "?";
}

int main(int argc, char *argv[]) {
  struct stat st;

  if (argc < 2) {
    fprintf(2, "usage: mystat name...\n");
    exit(1);
  }

  for (int i = 1; i < argc; i++) {
    if (stat(argv[i], &st) < 0) {
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
```

```
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }

  exit(0);
}
```

Listing 2.4 – user/mystat.c

The program was then used to watch hard links at work:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Listing 2.5 – mystat, ln and rm in xv6

Remark: `.` shows `links 2` and not 1 because the directory `a` had already been created in Part 1. In xv6 a new directory does not count its own `.` (the comment in `create()` says "No ip->nlink++ for '.': avoid cyclic ref count"), but its `..` entry increments the link count of the parent: `dp->nlink++; // for ".."`. On a fresh disk the root has 1 link.

Answers to the Observe questions

1. Paste the outputs after ln and after rm. What changed and what did not?

The outputs are in Listing 2.5. After `ln README r2` both names show the same inode 2, the same size 2441 bytes, and the link counter grew from 1 to 2: a hard link only adds a new directory entry pointing to the existing inode, no data is copied. After `rm README` only the name README disappeared and the link counter dropped back to 1; the inode number, the size and the content stayed the same – `cat r2 | wc` still counts 2441 bytes (48 lines, 336 words), the original README. This is exactly the behaviour observed on ext4 in Lab 5.

2. Find the line in kernel/sysfile.c (sys_unlink) that decrements the link count. When are the data blocks freed?

```
// kernel/sysfile.c, sys_unlink() (starts at line 202)
238    if (ip->type == T_DIR) {
239      dp->nlink--;          // the ".." of a removed directory
240      iupdate(dp);
241    }
242    iunlockput(dp);
243
244    ip->nlink--;            // <-- the link count of the file
245    iupdate(ip);
246    iunlockput(ip);

// kernel/fs.c, iput() (line 350)
357    int last = (ip->ref == 1 && ip->valid && ip->nlink == 0);
       ...
365      itrunc(ip); // free the data blocks
```

Listing 2.6 – Decrementing the link count and freeing the blocks

The link count is decremented at line 244, `ip->nlink--;`, after the directory entry has been overwritten with zeros, and `iupdate(ip)` writes the new value to the on-disk inode. The data blocks are not freed by `unlink` itself. They are freed in `iput()` (`kernel/fs.c`), which is called by `iunlockput()`: when the last in-memory reference is released (`ip->ref == 1`) and the inode has no more links (`ip->nlink == 0`), `iput` calls `itrunc()`, which releases all direct and indirect data blocks (`bfree`), and then `ifree()` marks the inode itself as free. So the blocks are freed only when the last name is removed **and** no process still has the file open – for README this did not happen, because r2 still linked to inode 2.

3. xv6 has no symbolic links and no permissions. Look at struct dinode in kernel/fs.h: which fields does an xv6 inode have?

```
struct dinode {
  short type;              // File type
  short major;             // Major device number (T_DEVICE only)
  short minor;             // Minor device number (T_DEVICE only)
  short nlink;             // Number of links to inode in file system
  uint size;               // Size of file (bytes)
  uint addrs[NDIRECT + 1]; // Data block addresses
};
```

Listing 2.7 – struct dinode in kernel/fs.h

An xv6 on-disk inode has only six fields: `type` (0 = free, or T_DIR / T_FILE / T_DEVICE), `major` and `minor` (which driver handles a device file), `nlink` (number of directory entries pointing to it), `size` (in bytes) and `addrs[13]` (12 direct block numbers and one indirect block). Compared with an ext4 inode it has no owner (uid/gid), no permission bits, no timestamps and no extents; this is why `chmod`, `chown` and symbolic links do not exist in xv6. The structure is $2 \cdot 4 + 4 + 13 \cdot 4 = 64$ bytes, so 16 inodes fit in one block.

2.4 Part 3. Measuring the limits

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

Listing 2.8 – The constants of kernel/fs.h and the disk layout printed by mkfs

```
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
averyveryveryl 1 27 32
cat            2 3 35376
...
```

Listing 2.9 – A directory with a name longer than DIRSIZ

Answers to the Observe questions

1. What does ls show for averyveryverylongname, and which constant explains it?

`ls` shows `averyveryveryl` – only the first 14 characters of the 21-character name. The constant is `DIRSIZ = 14`: the `name` field of `struct dirent` is a fixed array of 14 bytes, and the kernel (`dirlink`, via `strncpy`) copies at most 14 characters into it. A name of exactly 14 characters has no terminating zero, which is why `ls` and `find` must copy the name into a larger buffer before treating it as a C string. The truncation is silent: the directory can still be opened by the long name, because path lookup also compares only the first 14 characters (`namecmp` uses `strncmp(s, t, DIRSIZ)`).

2. Compute the largest file xv6 can hold.

An inode has `NDIRECT` = 12 direct block addresses and one indirect block. The indirect block is a block of `BSIZE` = 1024 bytes filled with 4-byte block numbers, so `NINDIRECT` = 1024 / 4 = 256 block addresses. Therefore:

```
MAXFILE = NDIRECT + NINDIRECT = 12 + 256 = 268 blocks
max size = 268 × BSIZE = 268 × 1024 B = 274 432 bytes = 268 KiB
```

Listing 2.10 – The maximum file size in xv6

This is the constant `MAXFILE` in `kernel/fs.h`; `writei()` refuses to write beyond `MAXFILE*BSIZE` bytes.

3. From the nmeta line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?

Table 2.1 – Layout of the xv6 disk image fs.img (blocks of 1024 bytes)

Region	Blocks	Explanation
boot block	1	block 0, unused by xv6
superblock	1	block 1, describes the layout (struct superblock)
log	31	LOGBLOCKS = MAXOPBLOCKS × 3 = 30 data blocks + 1 header block
inode blocks	13	NINODES = 200 inodes, 16 per block → ⌈200 / 16⌉ = 13
bitmap	1	one bit per block; 2000 bits fit in one block of 8192 bits
metadata total (nmeta)	47	1 + 1 + 31 + 13 + 1
data blocks	1953	for file and directory contents
whole disk (FSSIZE)	2000	47 + 1953 = 2000 blocks = 2 000 KiB (fs.img is 2 048 000 bytes)

So the whole disk has **2000 blocks**, of which 31 are for the log, 13 for the inodes and 1953 for data; the remaining 3 metadata blocks are the boot block, the superblock and the free-block bitmap.

2.5 Part 4. The program find

`find <dir> <name>` prints every path under `<dir>` whose last component is `<name>`. It was written starting from `user/ls.c`, which already shows how to open a directory, read `struct dirent` entries one by one, build the full path in a buffer and call `stat()` on it. The function `find()` works as follows:

- it opens the path and calls `fstat()`; if the path is a file or a device, it compares its last component (obtained with `fmtname()`) with the target;
- if the path is a directory, it copies the path into `buf`, appends `/`, and reads the directory 16 bytes at a time as `struct dirent`;
- entries with `inum == 0` are free slots and are skipped; `.` and `..` are skipped too, otherwise the recursion would never end (`.` points to the same directory and `..` goes back up);
- the name is copied after the `/` with `memmove(p, de.name, DIRSIZ)` and terminated with `p[DIRSIZ] = 0` (the name may have 14 characters and no zero); if it equals the target (`strcmp`), the full path is printed; if the entry is itself a directory, `find()` is called recursively on it;
- every directory is closed before returning, because an xv6 process can have only 16 open files (`NOFILE`); without `close()` a deep tree would exhaust the file table. A path that would not fit in the 512-byte buffer is reported and skipped.

```
#include "kernel/types.h"
#include "kernel/stat.h"
```

```
#include "user/user.h"
#include "kernel/fs.h"

char* fmtname(char *path) {
  char *p;
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;
  return p;
}

void find(char *path, char *target) {
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, 0)) < 0) {
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  switch (st.type) {
  case T_DEVICE:
  case T_FILE:
    if (strcmp(fmtname(path), target) == 0) {
      printf("%s\n", path);
    }
    break;

  case T_DIR:
    if (strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
      printf("find: path too long\n");
      break;
    }
    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';

    while (read(fd, &de, sizeof(de)) == sizeof(de)) {
      if (de.inum == 0)
        continue;

      if (strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
        continue;

      memmove(p, de.name, DIRSIZ);
      p[DIRSIZ] = 0;

      if (stat(buf, &st) < 0) {
        printf("find: cannot stat %s\n", buf);
        continue;
      }

      if (strcmp(de.name, target) == 0) {
        printf("%s\n", buf);
      }

      if (st.type == T_DIR) {
        find(buf, target);
      }
    }
    break;
  }
  close(fd);
}

int main(int argc, char *argv[]) {
```

```
  if (argc < 3) {
    fprintf(2, "usage: find <dir> <target>\n");
    exit(1);
  }

  find(argv[1], argv[2]);
  exit(0);
}
```

Listing 2.11 – user/find.c

The program was tested on a fresh disk (`make clean ; make qemu`), with the commands of the assignment and two extra checks (a name that does not exist, and the call without arguments):

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
$ find . nosuchfile
$ find
usage: find <dir> <target>
```

Listing 2.12 – Testing find

The results match the expected output of the assignment: all three files named `r2` are found at any depth, `find a b` finds the directory `a/b` itself, a missing name prints nothing, and a wrong call prints the usage message to standard error.

Stretch: which r2 is the same file as cat?

Instead of adding a `-n` option to `find`, the inode numbers were read with `mystat`:

```
$ mystat cat r2 a/r2 a/b/r2
cat: file, inode 3, links 2, size 35376 bytes
r2: file, inode 3, links 2, size 35376 bytes
a/r2: file, inode 29, links 1, size 2 bytes
a/b/r2: file, inode 28, links 1, size 2 bytes
```

Listing 2.13 – Inode numbers of the three r2 files

Only `./r2` is the same file as `cat`: both names point to inode 3, which has 2 links (it was created with `ln cat r2`), and its size is that of the program. `a/r2` (inode 29) and `a/b/r2` (inode 28) are different 2-byte files that only share the same name. Running `./r2` would therefore start the `cat` program.

CONCLUSIONS

In laboratory work no. 5 the Linux file system was examined directly from the terminal. The experiments showed that a file name is only a directory entry pointing to an inode: a hard link is a second entry for the same inode, `rm` only removes a name and decrements the link counter, and the data disappear only when the last link is gone. A symbolic link, on the other hand, is a small separate file that stores a path and becomes dangling when its target is removed. The difference between `ls -l` and `du` showed that the size of a file is not the space it uses: space is allocated in 4 KiB blocks, and a sparse file can have a size of 1 GB with no blocks at all.

The permission experiments showed how the kernel checks exactly one class of bits (owner, group or others), how `umask` removes bits from the default modes, and that on a directory `r` allows listing the names while `x` allows passing through it. SetUID (`passwd`) and the sticky bit (`/tmp`) solve two real problems: controlled access to a protected file and safe sharing of a world-writable directory. Finally, `lsblk`, `df`, `mount` and `/proc` showed how the disk is divided into partitions and file systems, and that the kernel exposes its own state as files generated on demand.

In laboratory work no. 6 the same ideas were found in the source code of xv6. Its file system uses the same concepts as ext4 – inodes, directories as tables of 16-byte entries, link counters, a log – but with very small, visible limits: names of at most 14 characters (`DIRSIZ`), files of at most 268 blocks (274 432 bytes), and a disk of 2000 blocks of which 47 are metadata. Reading `sys_unlink()` and `iput()` confirmed in code what Lab 5 showed in practice: unlink only decrements `nlink`, and the blocks are freed by `itrunc()` when the last link and the last reference disappear. The xv6 inode has no owner and no permission bits, which explains why xv6 has neither permissions nor symbolic links.

Two user programs were written: `mystat`, which made the effect of `ln` and `rm` visible through the inode number and the link counter, and `find`, which walks the directory tree recursively. Writing `find` showed the practical details of working with raw directory entries: skipping free slots and the `./..` entries, adding a terminating zero to names of 14 characters, and closing every directory to stay within the limit of 16 open files per process.

REFERENCES

5. Vasile Brînză, GitHub repository. [Online]. Available: https://github.com/kynexi/os-2026-VasileBrinza
6. MIT, "6.1810: Operating System Engineering – Lab: Xv6 and Unix utilities (find)." [Online]. Available: https://pdos.csail.mit.edu/6.1810/